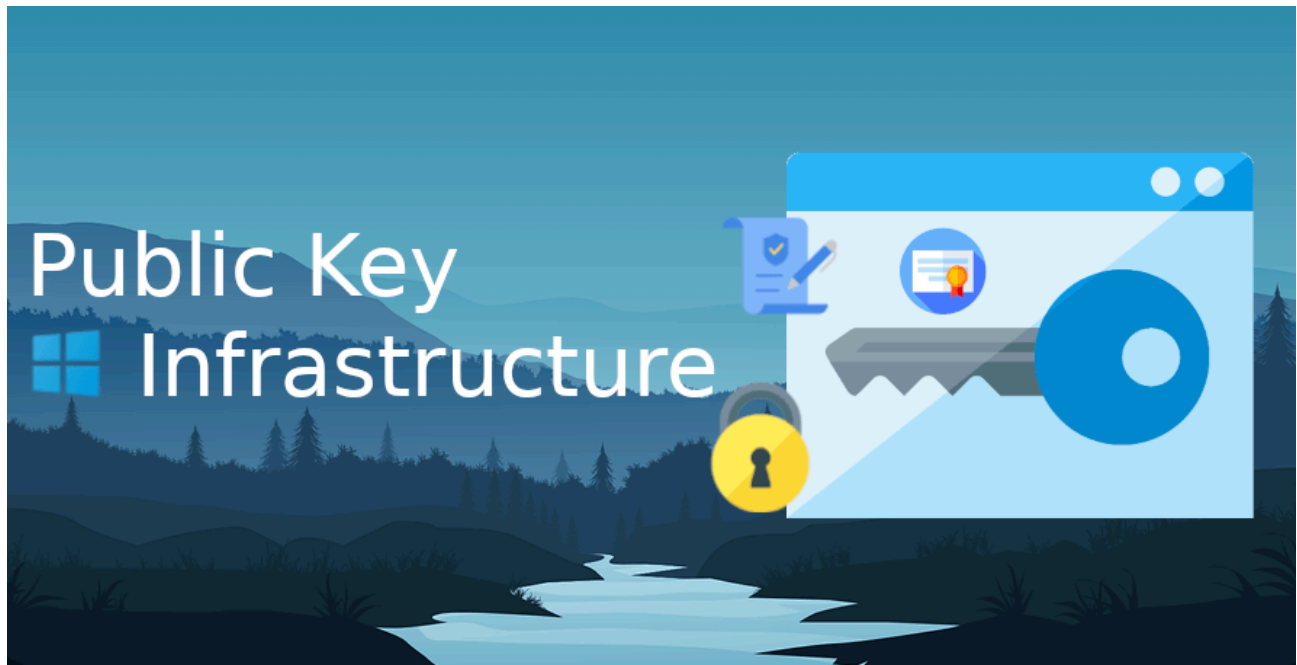


Build a PKI with PowerShell – Part 1 – Preparation – Michael Waterman

 michaelwaterman.nl/2025/12/31/how-to-build-a-pki-with-powershell-part-1-preparation

Michael Waterman

December 31, 2025



Other parts in this series

[How to: Build a PKI with PowerShell – Part 2 – IIS WebServer](#)

[How to: Build a PKI with PowerShell – Part 3 – Offline Root CA](#)

[How to: Build a PKI with PowerShell – Part 4 – Enterprise CA](#)

Over the last couple of years, I've written a lot about Public Key Infrastructure (PKI). Not the “*click next, next, finish*” type of posts, but the deeper stuff, why you'd pick one design over another, and what trade-offs you're really making.

Even so, I still see people struggling with PKI. Sometimes even setting up a relatively simple environment turns into a painful mix of conflicting guides, half-implemented best practices, and “set it and forget it” assumptions. The reality is: PKI quietly underpins almost everything we trust in modern IT environments, but it's often poorly documented, inconsistently implemented, and rarely treated like the living service it actually is.

PKI needs maintenance. Things expire, services evolve, and your environment changes. If you want to go deeper into the underlying concepts, feel free to check out my [PKI blog series](#) where I cover those fundamentals in more detail.

This series is a bit different. If you know me, you know I'm all-in on automation, because automation brings predictability. And predictability is exactly what you want when you're deploying security infrastructure. So in this "How-To", I'll walk through my approach to automating a two-tier, on-prem PKI setup.

I'll build a practical, production-ready PKI using PowerShell to keep the process consistent, repeatable, and auditable. This is not just a lab exercise. What I'm sharing is the result of many real-world implementations and lessons learned over the years. I'm not claiming this is the only way, or even the best way, but it is what has worked reliably for me and the customers I've supported.

The series is divided into four parts. This first part focuses on design decisions, preparation, and prerequisites. In the next parts, I'll build the environment step by step.

Note! *This series focuses on building PKI, not on certificate template design or day-to-day operations.*

What I'm building

The goal is a **two-tier PKI architecture**, consisting of:

- **An Offline Root Certificate Authority (CA)** Used only to sign subordinate CAs and kept offline for security.
- **An Enterprise Issuing CA** Used to issue certificates to domain-joined systems and services.
- **A Web Server**

Hosting:

- Certificate Revocation Lists (CRL)
- Certificate Distribution Points (CDP)
- Certification Practice Statement (CPS)

Why a two-tier PKI?

A single-tier CA is easy to deploy, and just as easy to compromise. It may look appealing at first, especially in smaller environments, but it quickly becomes a single point of failure. Once that root is compromised, trust is gone. Completely.

By separating trust (the Root CA) from daily operations (the Enterprise CA), you introduce a clear security boundary. The Root CA becomes a highly protected trust anchor, while the Enterprise CA handles the operational workload of issuing certificates to systems and services.

This design gives you several advantages:

- A strong and clearly defined security boundary
- Controlled and auditable certificate issuance
- Minimal exposure of the Root CA
- A clean recovery path when something inevitably goes wrong

The Root CA itself only needs to be online for very specific moments:

- Initial setup
- Renewal of subordinate CA certificates
- Publishing updated CRLs

Outside of those moments, it should remain powered off and securely stored. This drastically reduces the attack surface and limits the impact of potential compromises.

If you want to dive deeper into the *why* behind PKI design choices, including trust models, risk considerations, and long-term operational impact, I've covered that in more detail in an earlier blog post:

[PKI – Part 1: Introduction to Public Key Infrastructure](#)

Scope of this series

This blog series focuses specifically on the *technical implementation* of the PKI infrastructure:

- Private Enterprise Number (Pen) number
- Server preparation
- PowerShell-based installation
- Certificate hierarchy creation
- Validation and verification

It does **not** cover:

- PKI governance or policy writing
- Certificate lifecycle automation
- Cloud-based PKI or hybrid trust models

This is, simply put, the base for everything PKI based.

High-level architecture

At the end of this series, the environment will consist of:

- **IIS Web Server** Hosts CRL, CDP, and CPS endpoints.
- **Offline Root CA** Standalone, not domain-joined, highly restricted.

- **Enterprise CA** Domain-joined, responsible for issuing certificates to systems and services.

This design ensures a clean trust chain while keeping operational overhead manageable.

Prerequisites

Before touching PowerShell, some groundwork is required. These items should be defined **before** installation begins, including the .

Private Enterprise Numbers (PEN)

Before moving forward with the technical implementation, there's one often-overlooked prerequisite that deserves attention: *your Private Enterprise Number (PEN)*.

In short, a PEN is a globally unique identifier assigned by IANA (Internet Assigned Numbers Authority). It's used to uniquely identify an organization in various standards, including X.509 certificates, SNMP, and other security-related technologies. In the context of PKI, the PEN is typically used as part of your *Object Identifier (OID) namespace*.

In simple terms: your PEN ensures that anything you define within your PKI, certificate policies, custom extensions, or internal identifiers, is globally unique and will never collide with those of another organization.

Want to do a deeper dive? Check out my previous blog, [Part 5 – PKI Best Practices: Creating Unique Object Identifiers \(OIDs\)](#)

Why this needs your attention!

When designing a PKI, especially one intended for long-term use, uniqueness is critical. OIDs are embedded into certificates and may live for years, sometimes decades. Reusing generic or unofficial identifiers might work initially, but it introduces ambiguity, interoperability issues, and future maintenance challenges.

By registering your own PEN, you:

- Maintain full control over your internal OID structure
- Avoid conflicts with vendors or other organizations
- Ensure long-term compatibility and standards compliance
- Create a clean and extensible foundation for future certificate use cases

In short: this is one of those small steps that prevents big headaches later.

Obtaining a Private Enterprise Number

You can request a PEN for free (I'm Dutch so this matters to me) through IANA using the following link:

<https://www.iana.org/assignments/enterprise-numbers/assignment/apply/>

The process is straightforward and usually completed quickly.

Once assigned, your organization will appear in the public registry. For example, you can find my registered entry here: <https://www.iana.org/assignments/enterprise-numbers/?q=waterman>

My assigned Private Enterprise Number is, 55468

This number becomes the foundation for all custom OIDs used throughout the PKI configuration. No worries, I'll fill you in when and where you will use this number.

DNS configuration – PKI Web Server alias (trust)

Required permissions

Permission to create or modify DNS records in the target DNS zone (*typically DNS Admin or delegated DNS zone permissions*)

Why:

The PKI Web Server is accessed via a DNS alias (for example `trust.domain.suffix`) to provide a stable, clean endpoint for CRL, CDP, and CPS distribution. Creating this alias requires write access to the DNS zone hosting the record.

In most environments this means:

- Membership of the *DNS Admins* group, or
- Explicitly *delegated permissions* on the DNS zone

Domain Admin rights are *not strictly required* if DNS permissions are properly delegated. You'll need a DNS entry for your PKI endpoints:

- A-record for: `trust.<your-domain>.suffix`
- Optional PTR record for reverse lookups

Note! If you add a DNS alias, don't forget to align Kerberos authentication by configuring the corresponding SPNs. See [part two of this series](#) for the walk-through.

Minimum server requirements (high level)

Web Server

- 2 vCPU
- 6 GB RAM
- 60 GB disk
- Windows Server 2019 or newer
- Domain joined

Root CA

- 2 vCPU
- 2 – 4 GB RAM
- 60 GB disk
- Windows Server 2019 or newer
- Offline, **no** domain membership

Enterprise CA

- 2 vCPU
- 6 GB RAM
- 60 GB disk
- Windows Server 2019 or newer
- Domain joined

These specifications are intentionally low, PKI can hammer on your CPU depending on the load, scope accordingly.

Naming and configuration decisions

Before continuing, decide on:

- CA naming convention
- [Certificate lifetime](#)
- [Key lengths](#)
- CRL and CDP naming
- [Organizational identifiers \(OID\)](#)

These values are used consistently across all PowerShell commands and must not change once certificates are issued.

What's next?

[In Part 2 of this How to: Build a PKI with PowerShell series](#), we'll start building for real.

I'll deploy the PKI Web Server, configure IIS, and validate that CRL and CDP endpoints are reachable, a small step technically, but foundational for everything that follows.

References

[The Microsoft Root Certificate Program](#)

[PKI – Part 1: Introduction to Public Key Infrastructure](#)

[PKI – Part 2: Choosing the key length and algorithm](#)

[PKI – Part 3: The role of hash functions in PKI](#)

[PKI – Part 4: Understanding Cryptographic Providers](#)

[PKI – Part 5: Creating Unique Object Identifiers \(OIDs\)](#)

[PKI – Part 6: Demystifying the CAPolicy.inf file](#)